

HXC-159 04_deep_research

HXC-159 — §11.4.150 Deep Multi-Angle Research

Table of contents

Angle A — skill-system architecture and security

A1. Agent Skills is now an open, cross-vendor standard — this reframes the task

A2. Comparable models — and the structural contrast that matters most

A3. What mature plugin systems regret

A4. Security — the highest-value sub-angle

Angle B — GitHub SpecKit

B1. What it is, verified against the tool — not assumed

B2. The constitution collision — verified against the INSTALLED artifact

B3. Brownfield — better than the issue tracker suggests

Angle C — Go extension mechanics

C1. The plugin package is eliminated by its own documentation

C2. Decision matrix

C3. The shape-deciding Go finding — replace is ignored outside the main module

Angle D — the SpecKit ↔ Superpowers Bridge

D1. The Bridge design exists and is substantial — VERIFIED

D2. Layer-presence reality check — the middle of the Bridge is NOT installed

D3. Registration-path collision — now REAL, not hypothetical

D4. Constitution-derived extensions in scope — inventory

D5. Third-party dependency health — RETURNED, and the result is severe

5. The bigger underlying problem

Why this is the thing the team may not know

And this task is far past the measured cliff

The shape that survives all four angles

6. Honest gaps

Sources verified 2026-07-29

HXC-159 — §11.4.150 Deep Multi-Angle Research

Field	Value
Document	04_deep_research.md
Work item	HXC-159 (Task, In-progress, High)
Phase	1b — deep research, parallel to inventory (§11.4.150(F))
Revision	1
Last modified	2026-07-29
Author	(T1/main - claude1 - opus - xhigh)
Status summary	COMPLETE. 4 angles researched (A skill-system architecture · B SpecKit · C Go extension mechanics · D Bridge dependency health). Two findings reframe the task: skill shadowing (§5) and SuperBridge MCP does not exist (§D5).
Angles ≥ 2	Yes — 4 distinct angles, satisfying §11.4.150(B)
Anchors	§11.4.150 · §11.4.99 (latest sources) · §11.4.8 (cite or declare original) · §11.4.6 (FACT vs INFERENCE)

Table of contents

1. Angle A — skill-system architecture and security
2. Angle B — GitHub SpecKit
3. Angle C — Go extension mechanics
4. Angle D — the SpecKit ↔ Superpowers Bridge
5. The bigger underlying problem
6. Honest gaps
7. Sources verified

Every claim below is labelled **FACT** (cited to a source fetched 2026-07-29) or **INFERENCE** (reasoning from cited facts). Nothing is asserted from memory.

Angle A — skill-system architecture and security

A1. Agent Skills is now an open, cross-vendor standard — this reframes the task

FACT. The Agent Skills format was released by Anthropic as an **open standard** with a formal specification at `agentskills.io` and a governance repo at `github.com/agentskills/agentskills`. The client showcase lists roughly 45 independent implementations including **OpenAI Codex, Gemini CLI, GitHub Copilot, VS Code, Cursor, Amp, Goose, OpenHands, Roo Code, JetBrains Junie, Mistral Vibe, Kiro (AWS), Letta, Spring AI, Tabnine.** — <https://agentskills.io/> (accessed 2026-07-29)

Why this is decision-relevant. It converts "which vendor's skill format should HelixSkills target?" from a bet into a non-question, and it means a conformant skills corpus is portable to Gemini CLI / Qwen Code / Codex **for free** — which is exactly what §11.4.228's cross-agent compatibility mandate requires and what the §11.4.228 G1 gap says HelixCode currently lacks.

FACT — the complete normative frontmatter schema is 6 fields, only 2 required:

Field	Required	Constraints
name	Yes	1–64 chars, lowercase a-z0-9-; no leading/trailing -; no --; must match the parent directory name
description	Yes	1–1024 chars; must state <i>what</i> and <i>when</i>
license	No	name or bundled-file reference
compatibility	No	≤500 chars, free text
metadata	No	arbitrary string → string map
allowed-tools	No	space-separated; "Experimental. Support may vary between agent implementations."

Canonical layout `skill-name/{SKILL.md, scripts/, references/, assets/}`; validator `skills-ref validate ./my-skill`. Anthropic's platform doc adds two constraints the open spec omits: name cannot contain XML tags, and cannot contain the reserved words "anthropic" or "claude". — <https://agentskills.io/specification> · <https://>

platform.claude.com/docs/en/agents-and-tools/agent-skills/overview · <https://github.com/agentskills/agentskills/tree/main/skills-ref> (all accessed 2026-07-29)

Direct consequence for 06_risk_register.md R-05. The spec requires name to **match the parent directory name**, and the canonical file is SKILL.md. The measured constitution corpus violates both: three skills ship lowercase skill.md and multitrack ships neither. R-05 is therefore not a stylistic preference — it is **non-conformance with the published standard**, and the skills-ref validate tool exists to prove it mechanically.

FACT — progressive disclosure is a 3-tier contract with published costs. Level 1 (name+description) is **always loaded at startup, ~100 tokens per skill**. Level 2 (SKILL.md body) loads on trigger, <5k tokens / <500 lines recommended. Level 3 (bundled files) costs **zero until read**; scripts execute via bash and **only their output enters context, never their source** — so bundled context is "effectively unbounded." — <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills> (accessed 2026-07-29)

FACT — Claude Code extends the standard with 9+ proprietary fields (disable-model-invocation, user-invocable, disallowed-tools, argument-hint, arguments, context: fork, agent, background, plus \$ {CLAUDE_SKILL_DIR} substitution and !-prefixed dynamic shell context). Using them makes a skill non-portable. — <https://code.claude.com/docs/en/skills> (accessed 2026-07-29)

INFERENCE. A two-consumer library must declare a **conformance tier per skill** (portable spec-only vs Claude-Code-extended), or portability claims are unfalsifiable — a §11.4.6 no-guessing seam.

FACT — asymmetric lifetimes. Rendered SKILL.md content "stays there for the rest of the session" and "Claude Code does not re-read the skill file on later turns", while the allowed-tools grant "clears when you send your next message." — <https://code.claude.com/docs/en/skills> (accessed 2026-07-29)

INFERENCE. A test that edits a SKILL.md and re-invokes it *in the same session* validates the stale copy — a §11.4.108 SOURCE → RUNTIME trap specific to skill authoring. Skill tests must run in fresh sessions.

A2. Comparable models — and the structural contrast that matters most

FACT — MCP ships normative RFC-2119 security requirements (spec revision 2025-11-25): named attack classes Confused Deputy, Token Passthrough, SSRF, Session Hijacking, Local Server Compromise. "MCP servers **MUST NOT** accept any tokens that were not explicitly issued for the MCP server." "MCP Servers **MUST NOT** use sessions for authentication." One-click local server config "**MUST** implement proper consent mechanisms" showing "the exact command that will be executed, **without truncation**." — https://modelcontextprotocol.io/specification/2025-06-18/basic/security_best_practices (accessed 2026-07-29)

FACT — VS Code's model is process isolation plus deliberately minimal API: "A misbehaving extension cannot impact VS Code"; no DOM access because "extensions tightly coupled to the UI would break"; "Initially VS Code provides a small API surface... we will expand based on requests." Proposed APIs are usable in development but **cannot be published**. — <https://vscode-docs.readthedocs.io/en/stable/extensions/our-approach/> · <https://code.visualstudio.com/api> (accessed 2026-07-29)

FACT — Zed is the strongest sandboxing precedent: extensions are WASM components (wasm32-wasip1) in "a tightly controlled Wasmtime runtime", with a WIT-defined host/guest contract and a mechanical `wasm_api_version` range check. — <https://zed.dev/blog/zed-decoded-extensions> · <https://zed.dev/docs/extensions/developing-extensions> (accessed 2026-07-29)

INFERENCE — the decisive contrast. VS Code and Zed enforce their extension contract **mechanically** (process boundary / WASM sandbox + version-checked interface). Agent Skills enforce theirs **only by persuasion** — a skill is markdown telling a model what to do, executed with the agent's full ambient authority. There is no isolation boundary of any kind, and the Agent Skills spec contains **no security section at all**, where MCP ships MUST-level requirements. Every A4 finding is downstream of that single structural fact.

A3. What mature plugin systems regret

FACT — adoption is fast; deprecation cleanup is ~23× slower. ACM TOSEM, *The Co-evolution of the WordPress Platform and Its Plugins*: plugins declare support for a new platform version after a **median 38**

days, but delete uses of **removed** APIs a **median of 873 days** after removal. Newly added APIs are adopted "as fast as 1 day." — <https://dl.acm.org/doi/abs/10.1145/3533700> · https://sailresearch.github.io/sail-website/data/pdfs/2023_The_Co-evolution_of_the_WordPress_Platform_and_Its_Plugins.pdf (accessed 2026-07-29)

The lesson: a deprecation policy that assumes consumers will act is empirically false.

FACT — Manifest V3 is the canonical capability-drift case: announced November 2020, still forcing migrations in 2026; **runtime-fetched code banned outright** ("an extension can only execute JavaScript explicitly packed inside its zip and reviewed by the Chrome Web Store"); Mozilla diverged. — <https://developer.chrome.com/blog/resuming-the-transition-to-mv3/> (accessed 2026-07-29)

INFERENCE. MV3's core invariant — *no code that wasn't reviewed at publish time may execute* — is precisely what Claude Code's !-dynamic-context violates (A4.2). Chrome spent six years buying that invariant; the skills format gives it away by default.

A4. Security — the highest-value sub-angle

FACT — the first ecosystem-wide audit found a compromised supply chain. Snyk "ToxicSkills", 3,984 skills scanned from ClawHub + skills.sh as of 2026-02-05:

- **36.82 % (1,467)** carried security flaws
- **13.4 % (534)** carried at least one **critical** issue
- **36 %** contained **prompt injection**; **10.9 %** exposed hardcoded secrets
- **76 confirmed malicious payloads**, **8 still publicly available** at publication
- **91 % of malicious skills combined prompt injection with conventional malware**
- Observed: password-protected archives from attacker infrastructure, base64-obfuscated AWS-credential exfiltration, instructions to modify systemd units and delete system files
- **Registry bar to publish: "a SKILL.md Markdown file and a GitHub account that's one week old" — "No code signing. No security review. No sandbox by default."**

— <https://snyk.io/blog/toxicskills-malicious-ai-agent-skills-clawhub/>
(accessed 2026-07-29)

FACT — dynamic context executes shell BEFORE the model sees the skill, so model-level injection defences never engage: "Each dynamic context command executes immediately (**before Claude sees anything**)."

Weaponised PoC exfiltrates gh auth token via curl.

Mitigations named: "disableSkillShellExecution": true, review .claude/skills/ **before opening a project**, hunt for !`*curl and unrestricted Bash(*). — <https://securitylabs.datadoghq.com/articles/malicious-skills-supply-chain-risks-in-coding-agents-with-dynamic-context/> (accessed 2026-07-29)

FACT — allowed-tools is a privilege GRANT, not a sandbox.

Verbatim: it "grants permission for the listed tools... **without prompting you for approval... It does not restrict which tools are available: every tool remains callable.**" And: "**Review project skills before trusting a repository, since a skill can grant itself broad tool access.**" — <https://code.claude.com/docs/en/skills> (accessed 2026-07-29)

A skill in a repo is a self-service permission-escalation primitive.

Combined with the dynamic-context finding: allowed-tools: Bash(*) + !-commands = unprompted arbitrary RCE at file-read time.

FACT — Anthropic concedes there is no technical control, only trust: "a malicious Skill can direct Claude to invoke tools or execute code in ways that don't match the Skill's stated purpose"; "**Even trustworthy Skills can be compromised if their external dependencies change over time**"; "**Treat like installing software.**" — <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview> (accessed 2026-07-29)

FACT — real CVEs and a real in-the-wild attack already exist in adjacent MCP:

- **CVE-2025-49596** — Anthropic MCP Inspector, **CVSS 9.4** RCE; no auth between client and proxy; chained with "0.0.0.0 Day" + CSRF for code execution **merely by visiting a website**; 38k weekly downloads; fixed 0.14.1. — <https://www.oligo.security/blog/critical-rce-vulnerability-in-anthropic-mcp-inspector-cve-2025-49596>
- **CVE-2025-6514** — mcp-remote OS command injection; **558,846 downloads affected**; "first documented instance of full RCE on the client OS from a remote MCP server"; fixed ≥0.1.16. — <https://github.com/advisories/GHSA-6xpm-ggf7-wc3p>
- **postmark-mcp** — first malicious MCP server in the wild: attacker published **clean through v1.0.0–1.0.15 to build trust**, then added

an email-BCC backdoor in **v1.0.16 (2025-09-17)**; ~1,500 weekly downloads. — <https://thehackernews.com/2025/09/first-malicious-mcp-server-found.html>

(all accessed 2026-07-29)

The postmark-mcp timeline is the pattern to design against: 15 benign releases, then weaponisation. Pinning a *name* is worthless; only pinning a *content hash* defends.

FACT — OWASP LLM Top 10 (2025) mapping for governance traceability: **LLM01 Prompt Injection** (skill body is untrusted instruction text), **LLM03 Supply Chain** (expanded because "agents pull tools, frameworks, and other agents from external sources"), **LLM06 Excessive Agency** ("granted too much functionality, permissions, or autonomy"). OWASP now also ships an MCP Security Cheat Sheet. — https://cheatsheetseries.owasp.org/cheatsheets/MCP_Security_Cheat_Sheet.html (accessed 2026-07-29)

Angle B — GitHub SpecKit

B1. What it is, verified against the tool — not assumed

FACT. github/spec-kit, "Toolkit to help you get started with Spec-Driven Development." Requires Python 3.11+, uv/pipx, Git. Install/init:

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
specify init my-project --integration copilot
```

The flag is --integration <key>, **not** --ai. — <https://github.com/github/spec-kit> (accessed 2026-07-29)

FACT — the real command list is namespaced speckit. and longer than assumed. Core: /speckit.constitution, /speckit.specify, /speckit.plan, /speckit.tasks, /speckit.taskstoissues, /speckit.implement. Optional: /speckit.clarify, /speckit.analyze, /speckit.checklist. — <https://github.com/github/spec-kit> (accessed 2026-07-29)

INFERENCE — `/speckit.taskstoissues` is a **direct governance conflict**. It pushes tasks into **GitHub Issues**, which competes with HelixCode's §11.4.93/§11.4.95 SQLite-DB-as-single-source-of-truth for workable items. Adopting SpecKit without disabling or re-targeting this command forks the tracker.

FACT — **artifact layout verified:**

```
specs/[NNN-feature-name]/  
├─ spec.md   ── plan.md   ── tasks.md  
├─ research.md ── data-model.md  
├─ contracts/ ── quickstart.md  
.specify/{memory/constitution.md, scripts/, templates/}
```

— <https://raw.githubusercontent.com/github/spec-kit/main/spec-driven.md> (accessed 2026-07-29)

FACT — **SpecKit distributes its own commands AS SKILLS, as plain Markdown, into per-agent directories**. Claude Code → `.claude/skills`; Gemini CLI → `.gemini/commands`; Qwen Code → `.qwen/commands`; Codex → `.agents/skills`; Goose → `.goose/recipes/` (**YAML**); RovoDev → `.rovoDev/skills/` (**YAML**); plus a generic key with `--commands-dir.` — <https://github.github.io/spec-kit/reference/integrations.html> (accessed 2026-07-29)

Two consequences, both load-bearing. (i) This is a working, GitHub-official, 50-agent-scale **existence proof of the pure-data skills option** in Angle C — no code loading, no ABI, no linker. (ii) SpecKit and any HelixCode skills mechanism **compete for the same directory**, `.claude/skills`. This is no longer hypothetical — see the measured collision in Angle D.

B2. The constitution collision — verified against the INSTALLED artifact

The research literature and the installed reality differ here, and the difference matters. §11.4.6 requires reporting what was actually measured.

FACT (documentation). `spec-driven.md` describes a nine-article constitution including Article I "every feature must begin as a standalone library", Article III "no implementation code shall be written before unit tests are written", and Articles VII–VIII anti-over-

engineering "limiting initial projects to three maximum." — <https://raw.githubusercontent.com/github/spec-kit/main/spec-driven.md> (accessed 2026-07-29)

FACT (installed artifact, measured in this repo 2026-07-29). The file `specify init` actually writes to `.specify/memory/constitution.md` is a **2,346-byte placeholder template**, not the nine-article document:

```
# [PROJECT_NAME] Constitution
## Core Principles
### [PRINCIPLE_1_NAME]
<!-- Example: I. Library-First -->
[PRINCIPLE_1_DESCRIPTION]
```

The nine articles appear only as **HTML-comment examples**, not as prescribed content.

Correction, stated plainly (§11.4.6): the widely-repeated claim that SpecKit *imposes* a three-project cap and a library-first mandate is **not true of the installed artifact**. It ships a blank template whose examples suggest those patterns. The real risk is milder than the documentation implies — and reporting it as prescribed would have been a bluff.

The residual risk is still real, and is structural rather than doctrinal: a **third** constitution file now exists in this repo, and CONST-059 canonical-root ambiguity applies. Measured today:

Layer	Path	Size
Canonical root	<code>constitution/Constitution.md</code>	1,513,995 bytes
Consumer extension	<code>CONSTITUTION.md</code>	352,111 bytes
SpecKit (new)	<code>.specify/memory/constitution.md</code>	2,346 bytes

None of the three declares inheritance from another, and `/speckit.constitution` will rewrite the third without reference to the first two. There is **no documented mechanism to point SpecKit at an existing external constitution**.

B3. Brownfield — better than the issue tracker suggests

FACT. specify `init . / --here` work in an existing repo; issue #1285 (brownfield documentation) is **CLOSED**. — <https://github.com/github/spec-kit/issues/1285> (accessed 2026-07-29)

FACT. Issue #1436 "[Extension Proposal] Brownfield Bootstrap" — opened 2026-01-06, **still OPEN, no maintainer response** — names the gaps in the maintainers' own tracker: generic templates "failing to reflect actual project architecture"; manual constitution burden; "**no built-in support for multi-module / monorepo structures**"; no reverse-engineering from existing code. — <https://github.com/github/spec-kit/issues/1436> (accessed 2026-07-29)

FACT — counter-evidence. Discussion #1119 tested SpecKit across a genuine five-repo brownfield workspace: **minimal friction**; auto-discovered Go dependencies and logging patterns across repos; path references stayed accurate; did not create branches in implementation repos; asked before modifying. Single recommendation: "add information about the multi-repo workspace to the constitution." — <https://github.com/github/spec-kit/discussions/1119> (accessed 2026-07-29)

INFERENCE. The multi-repo story is better than the tracker suggests and the mitigation is cheap. The unmitigated risk is the **constitution/namespace collision**, not path resolution.

Angle C — Go extension mechanics

C1. The plugin package is eliminated by its own documentation

FACT, all from <https://pkg.go.dev/plugin> (accessed 2026-07-29):

- "Plugins are currently supported only on Linux, FreeBSD, and macOS, making them unsuitable for applications intended to be portable." → **no Windows.**
- "A plugin is only initialized once, and cannot be closed." → no unload, no hot-reload, no test isolation between loads.
- "Plugins are poorly supported by the Go race detector. Even simple race conditions may not be automatically detected." (<https://go.dev/issue/24245>)

- "Runtime crashes are likely to occur unless all parts of the program... are compiled using **exactly the same version of the toolchain, the same build tags, and the same values of certain flags and environment variables.**"
- "The application and its plugins must all be built together by a single person or component of a system."

That last clause is dispositive. A skills library shared between two *independently built* Go consumers is precisely the shape plugin documents as unsupported. And HelixCode's `make prod` cross-compile **Windows**, so plugin is eliminated on platform support alone before its other seven drawbacks are weighed.

FACT — corroborating production precedent. Kustomize marks Go plugins **deprecated**: "an end user accepting a shared plugin must compile both kustomize and the plugin"; "The only sensible way to share a plugin is as some kind of bundle ... containing source code." — https://kubectl.docs.kubernetes.io/guides/extending_kustomize/go_plugins (accessed 2026-07-29)

The recurring failure "plugin was built with a different version of package" spans Go 1.11 → 1.21; issue #27751 was **closed as not planned**. — <https://github.com/golang/go/issues/27751> (accessed 2026-07-29)

C2. Decision matrix

Option	 Windows	 Desktop (Fyne)	 Mobile	Survives go test	CGO
plugin	 No (FACT)			race- detector- blind, cannot unload (FACT)	UNCONFIRMED: 
hashicorp/ go-plugin	 (INFERENCE)		(INFERENCE  — iOS forbids subprocesses; unverified)		
wazero / extism	Yes (FACT:		(INFERENCE)		No CGO (FACT)

Option	Windows amd64	Desktop (Fyne)	Mobile	Survives go test	CGO
Compile-time registry					
Pure-data skills					

FACT — **wazero** is "the only zero dependency WebAssembly runtime written in Go", no CGO, Windows amd64 supported and CI-tested, deny-by-default host-function sandbox, used by Tetragon, Cilium, k6, Trivy, dapr. — <https://wazero.io/> · <https://github.com/tetratelabs/wazero> (accessed 2026-07-29)

FACT — **hashicorp/go-plugin**: gRPC/net-rpc over stdio; "Plugins can't crash your host process"; "The plugin only has access to the interfaces and args given to it"; "**no version matching required**" — the exact constraint plugin cannot escape. Powers Terraform, Vault, Nomad; "deployed on millions of machines." — <https://github.com/hashicorp/go-plugin> (accessed 2026-07-29)

FACT — **compile-time registry** is the "boring" option with std-lib precedent (database/sql drivers, image/png, net/http/pprof on blank import). Its one footgun — a forgotten blank import means `init()` never runs and registration silently never happens — has a documented mitigation: "adding validation in your `init()` that panics if expected plugins aren't registered." — <https://eli.thegreenplace.net/2021/plugins-in-go/> · <https://therealshek.medium.com/the-blank-import-in-go-when-side-effects-matter-more-than-names-32dab241b31e> (accessed 2026-07-29)

INFERENCE, directly relevant to §11.4.108. That footgun is *mechanically detectable*: a registry-completeness assertion at `init` or in a test is a §11.4.108 runtime signature. It converts "capability silently missing from a consumer" from an invisible failure into a hard startup failure. **None of the dynamic options offer an equivalently cheap check.**

C3. The shape-deciding Go finding — replace is ignored outside the main module

FACT, verbatim from the Go Modules Reference: "replace directives **only apply in the main module's go.mod file and are ignored in other modules.**" Identically for exclude. Version conflicts are resolved by MVS taking the higher version, and **there is no lock file.** — <https://go.dev/ref/mod> (accessed 2026-07-29)

INFERENCE — this is the mechanism behind "a capability lands in one consumer and not the other." If a shared skills library carries a replace, **neither consumer inherits it;** every consumer must independently restate every replace. A capability that works in the library's own tests can fail in a consumer for a reason invisible in the library's go.mod.

This is not hypothetical here. Measured in this repo 2026-07-29, submodules/helix_agent/go.mod carries **at least 10 replace directives:**

```
replace dev.helix.dag          => ../dag_orchestrator
replace digital.vasic.containers => ../containers
replace digital.vasic.challenges => ../challenges
replace digital.vasic.agentic   => ../agentic
replace digital.vasic.llmops    => ../llm_ops
replace digital.vasic.selfimprove => ../self_improve
replace digital.vasic.planning  => ../planning
replace digital.vasic.benchmark => ../benchmark
replace digital.vasic.llmsverifier => ../llms_verifier/llm-verifier
```

Every one is a relative path that resolves **only** when helix_agent is the main module. HelixCode consuming a shared library that itself depends on any of these would silently not inherit them.

FACT — go.work does not close the gap: "the go.work file is great for local development, but you probably do not want it in CI... CI builds should test each module independently using its declared dependencies in go.mod." — <https://oneuptime.com/blog/post/2026-02-01-go-workspaces-monorepos/view> (accessed 2026-07-29)

INFERENCE. Go's dependency model is **pull, never push.** Local go.work masks the divergence; CI/release un.masks it. That gap between local-green and release-real is a textbook §11.4.108
SOURCE → ARTIFACT divergence.

FACT — a repo-local hazard, flagged by research as unverified and now VERIFIED by me: the root and inner modules **both declare the same module path**:

```
go.mod:          module dev.helix.code    (go 1.25.2)
helix_code/go.mod: module dev.helix.code    (go 1.26)
```

Two modules sharing one module path, on different Go versions, in one repository. This is not the diamond problem above but is its own resolution hazard, and it sits directly under any shared-library seam.

Angle D — the SpecKit ↔ Superpowers Bridge

Added mid-flight per the binding operator requirement: *"SpecKit and Superpowers are used with Bridge extension and with all extensions we have created derived from constitution Submodule."*

D1. The Bridge design exists and is substantial — VERIFIED

FACT (local, measured 2026-07-29). constitution/docs/research/extensions/speckit_superpowers/implementation/ contains **8 Markdown documents**, each with .html/.pdf/.docx siblings (§11.4.65):

```
APPENDIX.md  CONSTITUTION_INTEGRATION.md  EXTENSION_DEVELOPMENT.md
IMPLEMENTATION_PLAN.md  NANO_TASK_ENGINE.md  README.md
SECURITY.md  TDD_INTEGRATION.md
```

plus request_with_materials.md one level up. EXTENSION_DEVELOPMENT.md alone is 92 KB; APPENDIX.md 79 KB. This is a real, sizeable design corpus, not a stub.

Catalogue-Check: reuse HelixDevelopment/constitution@ce3331a1 — the Bridge design is own-org, inherited by reference (§11.4.228), and HelixSkills **must compose with it rather than beside it**.

D2. Layer-presence reality check — the middle of the Bridge is NOT installed

This is the §11.4.108 SOURCE-vs-RUNTIME distinction the operator asked for. Probed directly on this host, in this checkout, 2026-07-29:

#	Layer	Probe	Result
L1	Developer workstation	—	present
L2	Spec-Kit Core	specify --version	specify 0.13.5.dev0 installed; .specify/ PRESENT ; specs/ ABSENT
L3	SuperSpec	command -v superspec	NOT INSTALLED
L4	SuperB	command -v superb	NOT INSTALLED
L5	SuperBridge MCP	grep .mcp.json	ABSENT
L6	Helix LLM	submodules/helix_llm	present
L7	llama.cpp	dependencies/LLama_CPP	present (submodule dir)

Verdict, stated plainly: layers 3, 4 and 5 — the entire orchestration, discipline and execution middle of the Bridge — are designed but NOT installed. L2 and L6/L7 are real. HelixSkills would therefore plug into a **partially paper pipeline**: the ends exist, the middle does not.

Honest boundary (§11.4.6): absence of a superspec/superb binary on PATH proves they are not installed *as CLI tools on this host*. It does not by itself prove the projects do not exist or could not be installed — that is what the Angle D third-party health research (in flight at time of writing) determines. What is proven here is the **runtime gap**, which is the decision-relevant fact.

D3. Registration-path collision — now REAL, not hypothetical

FACT (measured 2026-07-29). During this very session a sibling agent ran `specify init`, and `.claude/skills/` in this repo now contains *exactly 10 speckit- skills** created at 00:31:

speckit-analyze	speckit-checklist	speckit-clarify	speckit-constitution
speckit-converge	speckit-implement	speckit-plan	speckit-specify
speckit-tasks	speckit-taskstoissues		

SpecKit already occupies `.claude/skills/` — the exact namespace a HelixSkills registrar would write into. The collision predicted from Angle B is now observable in this checkout.

FACT — and the namespace is gitignored:

```
$ git check-ignore -v .claude/skills
.gitignore:133:.claude/*      .claude/skills
```

INFERENCE — two consequences. (i) The registered skills are **host-local untracked state**: a fresh clone has zero registered skills, so "the skills are installed" is unreproducible and unverifiable from the repository — a §11.4.77 regeneration-mechanism gap. (ii) Because the directory is ignored, a collision between SpecKit's 10 skills and any HelixSkills registration would leave **no diff in git status** — it would be silent.

FACT — a correction worth stating. The 10 speckit skills also exist at `/mnt/track1/atmosphere-t1/.claude/skills/`, i.e. in a **different project**. I did not read or modify anything under that path. The HelixCode registration is separate and was created in this session; earlier in this same session HelixCode's `.claude/` contained only `settings.json`, `settings.local.json` and `worktrees/`. So the registration is **new**, not pre-existing — which is itself the evidence that this namespace is actively contested.

D4. Constitution-derived extensions in scope — inventory

FACT (measured). Inherited by reference per §11.4.228, never copied:

Class	Count	Members
constitution/ skills/	7	action-prefix-system, media-validator, multitrack, reporting-workable-items, scheduled-work-queue, session-sync, workable-item-lifecycle
constitution/ mcp/	2	media-validator-mcp.json, scheduled-work-mcp.json
constitution/ plugins/	2	helix, scheduled-work
constitution/ actions/	—	registry.yaml (§11.4.140), subagent_tiering.yaml
constitution/ scripts/hooks/	7+	

Class	Count	Members
		live guards + libs (one blocked this session's first agent dispatch — proven live)

FACT — §11.4.164's auto-propagation is only half-wired in HelixCode. constitution/scripts/post_update_hook.sh **exists** (20,975 bytes, executable), but the consumer-side registrars it is specified to call **do not**:

```
$ ls scripts/register_skills.sh scripts/register_mcp.sh
ls: cannot access 'scripts/register_skills.sh': No such file or directory
ls: cannot access 'scripts/register_mcp.sh': No such file or directory
```

So on a constitution pull, skill/MCP registration in HelixCode has no consumer-side implementation to invoke. This is a pre-existing §11.4.164 gap that HXC-159 inherits and must close.

D5. Third-party dependency health — RETURNED, and the result is severe

#	Target	Verdict	Summary
L3	SuperSpec (WangX0111/superspec)	EXISTS	MIT, but 10 lifetime commits, bus-factor 1 , no push in ~8 weeks
L4	SuperB (speckit-community.github.io/extensions/superb)	EXISTS	Backing repo RbBtSn0w/spec-kit-extensions; genuinely active (12 releases); 1 human
L5	SuperBridge MCP	DOES NOT EXIST	Zero repos, zero packages. The name was explicitly rejected by the real project
—	Superpowers (obra/superpowers)	EXISTS, healthy	262k★, v6.2.0, pushed 2026-07-28
—	SpecKit extension system		

#	Target	Verdict	Summary
•		EXISTS and is OFFICIAL	...but both bridges sit in the un-audited community catalog

L5 SuperBridge MCP does not exist — the execution layer is fabricated

FACT — exhaustive negative search:

- api.github.com/search/repositories?q=superbridge+mcp → **total_count: 0**
- npm search superbridge (20 results) → **zero MCP servers**; the hits are an Electron IPC bridge, a crypto superchain bridge SDK, and a questdk plugin — all unrelated to agents, SpecKit, or MCP
- superpowers-bridge's own CHANGELOG: **"MCP" not found, "daemon" not found, "SuperBridge" not found**

FACT — the name is an explicitly *rejected* candidate. The real project's docs record: *"SuperB or SuperBridge should not be used as official public names for this extension... SuperBridge hides the product's relationship to Superpowers."* Corroborated first-hand by the v1.5.0 changelog entry documenting the naming hierarchy for Superpowers Bridge / superpowers-bridge / the superb namespace. (The quoted sentence is from a search-engine snapshot of the docs, not a byte-for-byte fetch — treat the *quote* as high-confidence-but-secondhand; the *changelog corroboration* is first-hand.)

FACT — L4 refuses the role the design assigns it. superpowers-bridge's README, verbatim: *"It owns no plan, task store, execution lifecycle, completion state, or convergence command."*

INFERENCE, high confidence. Three converging signals — no artifact of that name exists; the name was considered and *discarded* by the real project; and the real project explicitly disclaims owning any execution lifecycle — mean the design doc's layer 5 is **fabricated or aspirational**. *A 7-layer architecture whose execution tier is a non-existent MCP server is not an architecture; it is a diagram.*

Nearest real thing, and it is not a substitute. erophames/superpowers-mcp: real, TypeScript, **no LICENSE file**, 15★, 9 commits, **pushed 2026-03-15 (~4.5 months dead)**. Exposes 7 read-only tools. Security posture: does not execute arbitrary code, but **git-clones/pulls obra/superpowers via execFile and auto-pulls daily** — an unattended, unpinned, auto-updating fetch of remote instructions the agent then

executes, with **no sandboxing, no threat model, no license**. If layer 5 must be *something*, this is the closest candidate, and it is unlicensed and abandoned.

L3 SuperSpec — real, but the weakest link

FACT (api.github.com/repos/WangX0111/superspec): MIT, Shell, archived: false, created 2026-04-22, **pushed_at: 2026-06-02** (~8 weeks stale), 57★ / 7 forks / **0 watchers**. Full history is **10 commits by one human**; contributors API reports **1 contributor. 2 releases** — and v1.0.0 was **broken**, failing installation with "commands must use extension namespace"; v1.0.1 (2026-05-30) fixed the id.

FACT — its README's headline install command does not work on a default install. README recommends `specify extension add superspec`, but superspec is **not** in spec-kit's first-party extensions/catalog.json (4 entries, all maintainer: spec-kit-core). The working form requires an explicit archive URL: `specify extension add superspec --from https://github.com/WangX0111/superspec/archive/refs/tags/v1.0.1.zip`. A third documented path symlinks it into `~/.claude/skills/superspec`.

Supply-chain reading, blunt. One maintainer, ten lifetime commits, zero watchers, install by **mutable tag-archive URL** from a personal account — no checksum, no signature, no immutability. Shell scripts plus a symlink into the personal skills namespace is arbitrary-instruction injection into every future agent session. If the account is compromised or the author walks, there is no succession path.

L4 SuperB — the healthiest bridge, still bus-factor 1

FACT (api.github.com/repos/RbBtSn0w/spec-kit-extensions): MIT, Shell, created 2026-03-30, **pushed 2026-07-03**, 28★, **0 open issues. 12 tagged releases** (v1.0.0 → v1.9.0), five of them inside five weeks. Contributors: **RbBtSn0w (60) + two bots he owns**. Ships a **tests/directory** — the only bridge that does — and installs from an **immutable release .zip asset**, a real improvement over L3.

speckit-community is one person, not an organisation

FACT. api.github.com/orgs/speckit-community → **404**. api.github.com/users/speckit-community → **type: User**, created 2026-03-31, **4 public repos, 3 followers**, no name/company/bio. The catalog repo has **1 contributor** (ismaelJimenez, 9 commits).

FACT — the site's own disclaimer, verbatim: *"This website is a community-maintained catalog of Spec Kit extensions. It is not hosted, maintained, or affiliated with GitHub, Inc."*

INFERENCE. It is a **personal account styled to read like an organisation**. Not deceptive — the disclaimer is prominent — but a design doc citing `speckit-community.github.io` as an authority is citing one individual with three followers.

SpecKit's extension system is official; catalog listing is NOT an audit

FACT. `spec-kit` ships a real extension system (`RFC-EXTENSION-SYSTEM.md` 61 KB, API reference, `dev/publishing/user guides`, `specify extension search|add|remove|list`). Both bridges are listed in `catalog.community.json / docs/community/extensions.md` (SuperB landed via issue #2973 → PR #2998, merged 2026-06-16).

FACT — spec-kit's own FAQ, verbatim: *"The Spec Kit maintainers do not review, audit, endorse, or support extension code. Review an extension's source code before installing and use at your own discretion."* And `docs/community/extensions.md`: maintainers *"only verify that catalog entries are complete and correctly formatted — they do **not** review, audit, endorse, or support the extension code itself."*

INFERENCE. Listing is **metadata-schema validation, explicitly not a code audit**. "It's in spec-kit's catalog" is the wrong trust signal, and any design resting on it is reading a formatting check as a security review.

Superpowers — the one genuinely healthy dependency, and it de-risks collisions

FACT (`api.github.com/repos/obra/superpowers`): MIT, created 2025-10-09, **pushed 2026-07-28, 262,632★ / 23,453 forks / 987 watchers**, v6.2.0 released 2026-07-24 with sustained multi-release-per-month cadence. Distributed as a **plugin** through Anthropic's official marketplace (`/plugin install superpowers@claude-plugins-official`), not as loose `~/.claude/skills/` files.

(\$11.4.6 caveat: 262k★ on a 9-month-old repo is extraordinary. That is the value the GitHub API returned today; it was not independently corroborated.)

FACT — the standard-relationship framing is backwards in common retellings. Claude Code docs, verbatim: "*Claude Code skills follow the Agent Skills (agentskills.io) open standard.*" obra/superpowers' README does **not** mention agentskills.io at all.

INFERENCE (well-supported). They do not compete and need no conversion — they compose. Superpowers is a set of SKILL.md files, exactly what the standard specifies; it predates the standard's publication (repo 2025-10-09 vs standard 2025-12-18) and simply never re-badged.

Skill-namespace collision — better documented than assumed

FACT — Claude Code docs, verbatim: "*When skills share the same name across levels, enterprise overrides personal, and personal overrides project... **Plugin skills use a plugin-name:skill-name namespace, so they cannot conflict with other levels.***" Nested dirs are the exception that does *not* last-writer-win: "*If a nested skill shares a name with another skill, **both stay available.***" And symlinks de-duplicate: "*if the same target is reachable from more than one location, Claude Code loads the skill once.*"

FACT — the documented gap, raised and refused. anthropics/claude-code issue #50486 asked for plugin-name prefixing of plugin skills; **CLOSED AS NOT PLANNED**, no maintainer reply. Related open reports #25209 (project-level skills showing *both* instead of overriding — observed behaviour contradicting the documented rule) and #15065.

INFERENCE — three consequences for HXC-159. (i) Claude Code **does not error and does not warn** on a skill name collision; the failure mode is *silent shadowing*, not a diagnostic. (ii) The collision risk in *this* design is **smaller than feared**, because Superpowers installs plugin-namespaced ("cannot conflict") and SuperB installs as spec-kit / speckit.superb.* commands rather than .claude/skills/ entries — only SuperSpec's optional symlink path touches the personal namespace, and it claims one distinctive name. (iii) **NO external solution found – original work** (§11.4.8) for the case that actually bites here: **two independent installers both writing a directory of the same name into one .claude/skills/ tree.** That is a *filesystem* collision resolved before Claude Code ever reads it, and nothing upstream addresses it — install-time collision detection must be original work.

Supply-chain summary (§11.4.74 no-match → vendor)

Dependency	Maintainers	Last push	Releases	Pinnable?	Risk
obra/ superpowers	large, 987 watchers	2026-07-28	v6.2.0, monthly+	Yes — official marketplace plugin	LOW
RbBtSn0w/ spec-kit- extensions (SuperB)	1 human + 2 own bots	2026-07-03	12 , immutable.zip	Yes — release asset	MEDIUM
WangX0111/ superspec	1 human , 10 commits	2026-06-02	2 (v1.0.0 broken)	Weak — mutable tag archive	HIGH
speckit- community catalog	1 human , personal acct as "org"	2026-07-28	n/a	n/a	MEDIUM
erophames/ superpowers- mcp	9 commits, no LICENSE	2026-03-15 (~4.5 mo dead)	none	No	HIGH — auto-pulls remote instructions daily, unsandboxed
SuperBridge MCP	—	—	—	—	N/A — does not exist

Blunt reading. Three of the four named third-party layers are **single-maintainer projects under four months old**, and the one that matters most architecturally — the execution layer — **is not a project at all**. Two ship **Shell** and install by extracting a remote archive into a directory the agent then executes from, un-audited by GitHub *by GitHub's own explicit statement*. Only obra/superpowers clears an enterprise dependency bar, and only because it ships through Anthropic's official marketplace.

A methodological note worth preserving (§11.4.6). The research agent's first pass fetched the raw 182 KB catalog.community.json and reported "NOT FOUND" for all four names — **that read was truncated mid-file at an entry alphabetically before s, making the negative invalid**. The correct FOUND results came from reading docs/

community/extensions.md in full. A truncated-fetch false-null is exactly the §11.4.201 false-negative class, and it was caught and logged rather than shipped.

5. The bigger underlying problem

§11.4.150(C) requires explicitly hunting for a more serious problem than the stated task. **One was found, and it is peer-reviewed and quantified.**

FACT — skill libraries get measurably WORSE as they grow.

Databricks, *More Skills, Worse Agents? Skill Shadowing Degrades Performance When Expanding Skill Libraries* (Hongwen Song, Song Wei; arXiv:2605.24050v1, 2026-05-21). SkillsBench: 38 oracle (task, model) pairs, 2,545 trajectories, Claude Haiku 4.5 + Sonnet 4.6:

Library size	Pass-rate change
52 skills	-8 %
102 skills	-14 %
202 skills	-21 %

And the decisive detail: **"Skill shadowing accounts for up to 68 % of degradation at maximum library size, while context overhead remains statistically indistinguishable from zero."** Failure mode is model-dependent — Haiku *abandons* (selects no skill, -26 % at 202); Sonnet *mis-selects* (-15 %). — <https://arxiv.org/html/2605.24050v1> (accessed 2026-07-29)

Corroborating independent measurement: tool-selection accuracy "drops from above 90 % with fewer than 30 candidates to 13.6 % with 11,100 options." — <https://www.asikaibrain.com/en/posts/context-engineering-why-600-skills-make-agents-less-effective/> (accessed 2026-07-29)

Why this is the thing the team may not know

The intuitive fear about a large shared skills library is **token cost**. The measurement says token cost is **statistically zero** and the real damage is **semantic interference between overlapping description fields**. Therefore the intuitive mitigations — trimming prose, shortening

SKILL.md, lazy-loading bodies — **fix the wrong variable**. Progressive disclosure (A1) already solved context; it does nothing for shadowing, because Level-1 descriptions are *always* loaded and *always* compete.

And this task is far past the measured cliff

05_catalogue_survey.md measured **1174 SKILL.md files** in submodules/helix_agent/skills/. The study's worst measured case is **202 skills at -21 %**. The corpus HXC-159 would union is **~5.8× beyond the worst point ever measured**, and the curve is monotonically worsening across every point sampled.

INFERENCE, high confidence. "Fully incorporate... all power-features, nothing left out", read as *register every skill into one namespace*, is **the exact configuration the literature says degrades the agent**. Executing the task as literally worded would likely make both consumers measurably worse at their jobs while every test stayed green — because pass-rate degradation of this kind is invisible to unit tests and shows up only in end-to-end task success.

This is a §11.4.6 honest boundary, not a refusal: the study measures Claude Haiku 4.5 / Sonnet 4.6 on SkillsBench, not HelixCode on its own corpus. The effect is established; its exact magnitude *here* is not, and must be measured on this project's own fixtures before any threshold is chosen (§11.4.6 — thresholds calibrated on own fixtures, never borrowed from literature).

The shape that survives all four angles

Each clause traces to a specific cited finding, not to preference:

Design clause	Forced by
Per-consumer opt-in subsets , never whole-library mounting	A5.1 shadowing curve; 1174 » 202
Namespaced (/lib:skill), never flattened	A5.1 + the spec's name-matches-directory rule
Content-hash pinned , never name-pinned	postmark-mcp: 15 benign releases, then backdoor
Spec-conformance tiered (portable vs Claude-extended)	A1.4 proprietary-field divergence
allowed-tools + !-dynamic-context as review-gated privileged constructs	A4.2 + A4.3 — RCE at file-read time

Design clause	Forced by
Pure-data skills, not a Go plugin ABI	C1 (plugin eliminated) + B (SpecKit's 50-agent existence proof)
Registry-completeness assertion as the runtime signature	C2 — the only cheap mechanical parity check
Version-parity gate across consumers	C3 — replace ignored outside main module

6. Honest gaps

Per §11.4.6, stated rather than papered over:

- **Angle D is now COMPLETE**, and its central result is negative: **SuperBridge MCP does not exist**. The "SuperBridge" quote establishing the name was *rejected* is from a search-engine snapshot rather than a byte-for-byte fetch — high-confidence but secondhand; the changelog corroboration is first-hand.
- **obra/superpowers' 262k★** is the value the GitHub API returned today; it was not independently corroborated and is extraordinary for a 9-month-old repo.
- **anthropics/claude-code issues #25209 and #15065** were surfaced by search and not individually fetched; they are cited as *reported* behaviour, not verified.
- **ACM TOSEM primary returned HTTP 403**. The 38-day / 873-day figures come from the indexed abstract plus the SAIL open-access PDF. A "431 days" figure that appeared in one search snippet **did not verify** and is not used.
- **NO external solution found – original work** (§11.4.8) for: published guidance on **sharing one skills library across two agent consumers**. Vendors document only single-consumer distribution. The §5 design shape is therefore *inference from cited primitives*, not a cited pattern — and must be labelled as such wherever it is reused.
- **NO external solution found – original work** (§11.4.8) for: any **skill provenance/signing** scheme. Snyk's "No code signing. No security review. No sandbox by default." is the state of the art ecosystem-wide, not a registry-specific lapse.
- **NO external solution found – original work** (§11.4.8) for: a **Go-specific pure-data skill-interpretation framework** with

published semantics. The pattern is widely deployed; no library standardises it for Go hosts.

- **plugin + CGO coupling is UNCONFIRMED:** — widely asserted in secondary sources, no primary citation found. It does not change the verdict; the platform list and the "built together by a single person" clause already decide it.
 - **iOS subprocess prohibition is INFERENCE, unverified against Apple docs** — it is the basis for the "mobile" cell for hashicorp/go-plugin in C2.
 - **specifier's actual capability is unverified** beyond its root tree (see 05_catalogue_survey.md §7).
 - **Not fetched:** a comparative "Spec Kit vs BMAD vs OpenSpec" piece. If the decision becomes *which* SDD framework, that is the next fetch — no claim is made about its content.
-

Sources verified 2026-07-29

Agent Skills / security

- <https://agentskills.io/> · <https://agentskills.io/specification> · <https://github.com/agentskills/agentskills>
- <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>
- <https://code.claude.com/docs/en/skills>
- <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>
- <https://snyk.io/blog/toxicskills-malicious-ai-agent-skills-clawhub/>
- <https://securitylabs.datadoghq.com/articles/malicious-skills-supply-chain-risks-in-coding-agents-with-dynamic-context/>
- <https://www.oligo.security/blog/critical-rce-vulnerability-in-anthropic-mcp-inspector-cve-2025-49596>
- <https://github.com/advisories/GHSA-6xpm-ggf7-wc3p>
- <https://thehackernews.com/2025/09/first-malicious-mcp-server-found.html>
- https://cheatsheetseries.owasp.org/cheatsheets/MCP_Security_Cheat_Sheet.html

Comparable extension models

- https://modelcontextprotocol.io/specification/2025-06-18/basic/security_best_practices
- <https://modelcontextprotocol.io/specification/2025-06-18/basic/transports>

- <https://vscode-docs.readthedocs.io/en/stable/extensions/our-approach/> · <https://code.visualstudio.com/api>
- <https://zed.dev/blog/zed-decoded-extensions> · <https://zed.dev/docs/extensions/developing-extensions>
- <https://developer.chrome.com/blog/resuming-the-transition-to-mv3/>
- <https://dl.acm.org/doi/abs/10.1145/3533700> (403; via SAIL open-access PDF)

SpecKit

- <https://github.com/github/spec-kit> · <https://raw.githubusercontent.com/github/spec-kit/main/spec-driven.md>
- <https://github.github.io/spec-kit/reference/integrations.html>
- <https://github.com/github/spec-kit/issues/1285> · [/issues/1436](https://github.com/github/spec-kit/issues/1436) · [/discussions/1119](https://github.com/github/spec-kit/discussions/1119)

Go extension mechanics

- <https://pkg.go.dev/plugin> · <https://go.dev/issue/24245> · <https://github.com/golang/go/issues/27751>
- <https://go.dev/ref/mod>
- https://kubectldocs.kubernetes.io/guides/extending_kustomize/go_plugins
- <https://github.com/hashicorp/go-plugin>
- <https://wazero.io/> · <https://github.com/tetratelabs/wazero> · <https://github.com/extism/go-sdk> · <https://github.com/knqyf263/go-plugin>
- <https://eli.thegreenplace.net/2021/plugins-in-go/> · <https://therealshek.medium.com/the-blank-import-in-go-when-side-effects-matter-more-than-names-32dab241b31e>
- <https://oneuptime.com/blog/post/2026-02-01-go-workspaces-monorepos/view>

Bridge dependency health (Angle D)

- <https://github.com/WangX0111/superspec> · <https://api.github.com/repos/WangX0111/superspec> (+ /commits, /releases, /contributors)
- <https://speckit-community.github.io/extensions/superb> · <https://speckit-community.github.io/extensions/>
- <https://api.github.com/users/speckit-community> · <https://api.github.com/repos/speckit-community/extensions/contributors>
- <https://api.github.com/repos/RbBtSn0w/spec-kit-extensions> (+ /releases, /contributors)
- <https://raw.githubusercontent.com/RbBtSn0w/spec-kit-extensions/main/superpowers-bridge/README.md> + /CHANGELOG.md
- <https://github.com/obra/superpowers> · <https://api.github.com/repos/obra/superpowers> (+ /releases)

- <https://github.com/github/spec-kit/blob/main/docs/reference/extensions.md> · <https://raw.githubusercontent.com/github/spec-kit/main/extensions/catalog.json> · <https://raw.githubusercontent.com/github/spec-kit/main/docs/community/extensions.md>
- <https://github.com/github/spec-kit/issues/2973> · <https://github.com/github/spec-kit/pull/2998>
- <https://github.com/anthropics/claude-code/issues/50486> · [/issues/25209](#) · [/issues/15065](#)
- <https://github.com/erophames/superpowers-mcp> · <https://registry.npmjs.org/@chenmk/superflow>
- <https://api.github.com/search/repositories?q=superbridge+mcp> (→ total_count: 0)

The bigger problem

- <https://arxiv.org/html/2605.24050v1>
- <https://www.askaibrain.com/en/posts/context-engineering-why-600-skills-make-agents-less-effective/>

Local measurements (this session, 2026-07-29): specify --version; command -v superspec/superb; .mcp.json grep; ls .claude/skills; git check-ignore -v .claude/skills; ls .specify/memory/constitution.md + head; find constitution/docs/research/extensions/speckit_superpowers -name '*.md'; ls constitution/scripts/post_update_hook.sh; ls scripts/register_*.sh; grep '^module' go.mod helix_code/go.mod; grep replace submodules/helix_agent/go.mod.